Release 1.2.0 — Deployment Runbook (Manual Coolify Steps)

This release ships in **green, deployable increments** pushed to `main`. After Coolify redeploys `main`, perform the manual step(s) for each increment below. Increments are additive and non-destructive; existing sites keep working and do **not** need to reconnect.

> **How to open the API container shell:** In Coolify → your `marqira-api` application → **Terminal** (a.k.a. Console / Exec). You should get a shell inside the running container. All `php artisan ...` commands below run there.

Increment 1 — Platform roles, ownership, revocation & duplicate prevention

Commit: `492a6ce`
Risk: Low. Migrations are additive (new nullable columns + one partial unique index) and reversible. No data is deleted.

What changed (operationally)

- New `users.platform_role` (`owner` / `subscriber`) and `users.is_active`.
- Existing organization **owners are auto-promoted** to platform Owner during migration, so current admins keep full visibility. `ozman.best@gmail.com` is always promoted if present.
- New `sites.owner_user_id`, `sites.domain_normalized`, `sites.revoked_at`, `sites.revoked_by`. Ownership is backfilled from the enrollment token that created each site (where known).
- A **partial unique index** enforces one active site per (organization, normalized domain). Before the index is created the migration **soft-revokes** any pre-existing active duplicates, keeping the most recently-seen row (nothing is deleted).

Manual steps after redeploy

1. **(Recommended) Preview duplicate cleanup — safe, read-only:**
   `php artisan marqira:deduplicate-sites --dry-run`
   This lists every domain that has more than one active site and which rows would be soft-revoked. It writes nothing. Review the output; if a wrong row would be kept, resolve it in the dashboard first.

2. **Run migrations:**
   `php artisan migrate --force`
   Expected new migrations:
   - `2024_01_02_000001_add_platform_role_to_users`
   - `2024_01_02_000002_add_owner_and_revocation_to_sites`

- `2024_01_02_000003_add_sites_domain_unique_index`
- `2024_01_02_000004_create_account_invitations_table`

> The `..._add_sites_domain_unique_index` migration performs the safe soft-revoke of any remaining active duplicates automatically before adding the index, so it is safe even if you skipped step 1.

1. **Verify the primary Owner:**
   `php artisan tinker --execute="echo \App\Models\User::where('email','ozman.best@gmail.com')->value('platform_role');"`
   Should print `owner`. If that account does not exist yet, create it with `php artisan marqira:create-admin` (it is created as an Owner).
2. **(Optional) Re-run the dedup command without `--dry-run`** only if you later discover duplicates that were enrolled before this release and were not caught by the migration:
   `php artisan marqira:deduplicate-sites`

Rollback

If needed, `php artisan migrate:rollback` reverses these four migrations (drops the added columns/index/table). No site rows are removed by rolling back, but any sites soft-revoked during dedup will remain revoked — reactivate them from the dashboard if required.

No config/env changes required

This increment introduces **no** new environment variables. Email/SMTP settings are introduced in a later increment (offline alerting) and are documented then.

Increment 2 — Connector lifecycle: persistent pairing, cron self-heal & self-disconnect on revocation

Plugin version **1.2.0** (display name **MarQira Pulse**; folder/slug/prefix unchanged at `marqira-connector`). This increment pairs the WordPress connector with the revocation support shipped in Increment 1.

What changed (operationally)

- **Self-disconnect on revocation.** When a site is revoked from the dashboard, the API already answers its next heartbeat with `HTTP 403 {"error":"site_revoked","site_revoked":true}` (Increment 1). The 1.2.0 connector now detects this and **stops its heartbeat cron and clears its stored credentials**, so a revoked site goes quiet immediately instead of retrying a rejected beat every couple of minutes. Reconnecting requires a fresh enrollment code.
- **Plain 403s are safe.** A 403 that is not a revocation signal (e.g. a transient WAF or permission block) is logged as an ordinary failure and does **not** wipe credentials — a healthy site is never silently disconnected.
- **Persistent pairing across updates (confirmed).** Credentials are stored in the `marqira_site_credentials` option and preserved on deactivate/upgrade. Combined with the existing cron self-heal, updated sites keep monitoring with

no reconnection. (Increment 4 hardens this further so pairing also survives a full **delete + reinstall** — see below.)

- **“Updates available” now works out of the box.** The API config `marqira.plugin.latest_version` now defaults to `1.2.0` (env `MARQIRA_PLUGIN_LATEST_VERSION`), so the dashboard flags any site still running an older connector.

Manual steps after redeploy

1. **API:** No migrations. **Optional but recommended** — set (or confirm) the env var so the “Updates available” card reflects the shipped connector: `MARQIRA_PLUGIN_LATEST_VERSION=1.2.0`
   If the variable is left **blank**, Laravel treats it as empty and the card shows 0. If the variable is **absent**, the new `1.2.0` config default applies. After changing env in Coolify, redeploy (or run `php artisan config:cache`) so the value is picked up.
2. **Connector plugin:** Publish the updated `marqira-connector` folder (v1.2.0) to customers as a normal plugin update (same folder/slug). **No customer reconnection is required.** Existing enrolled sites keep their credentials and simply gain the self-disconnect behavior.

Verifying revocation end-to-end (optional)

1. Revoke a test site from the dashboard (Increment 1 `DELETE` / disconnect).
2. Within one heartbeat interval (~2 min test cadence) the site’s next beat receives `403 site_revoked`; the connector clears credentials and unschedules its cron. In **Settings → MarQira Connector → Recent Activity** you will see a `site_revoked` log entry.

Rollback

- **API:** revert the `marqira.plugin.latest_version` default (or unset the env var). No migrations were added, so there is nothing to roll back on the DB.
- **Connector:** re-publishing the previous 1.1.3 plugin folder restores the old behavior; enrolled sites are unaffected either way (credentials persist).

No destructive changes

No database migrations, no schema changes, and no forced reconnection. Existing sites continue heartbeating exactly as before, plus the new revocation handling.

Increment 3 — Offline monitoring with repeated & recovery email alerts

Commit: `cb416cd`
Risk: Low. One additive, reversible migration (three new nullable/default columns on `sites`). No data is deleted. New outbound email — **requires SMTP env + a running queue worker** (see manual steps).

What changed (operationally)

- **Server-driven offline detection with alerts.** The existing `marqira:check-stale-sites` scheduler now, in addition to marking a stale site

`offline`, **emails the site owner** (and the optional platform alert address) that the site is down. (Increment 4 changes this scheduler from every-5-minutes to **every minute** so short repeat intervals are honored — see below.)

- **Repeated alerts while down.** While a site stays offline, a reminder email is re-sent every `MARQIRA_OFFLINE_ALERT_REPEAT_MINUTES` (default 60). Set to `0` to send only the single initial alert.
- **Recovery alert.** When an offline site sends its next heartbeat, the API emails a "back online" recovery notice (only if at least one offline alert was sent during the outage) and resets the offline tracking.
- **No false alarms.** Revoked sites are never alerted (their connector was told to disconnect). A freshly enrolled site that has never sent a heartbeat is marked offline for status accuracy but does **not** trigger an email.
- **Emails are queued** (Redis queue) so neither the scheduler nor the heartbeat request blocks on SMTP.

New columns on `sites` (migration `2024_01_03_000001`)

- `offline_since` (nullable timestamp) — start of the current offline episode.
- `last_offline_alert_at` (nullable timestamp) — when the last alert was sent.
- `offline_alert_count` (unsigned int, default 0) — alerts sent this episode.

Manual steps after redeploy

1. **Set SMTP + alert env vars** in the Coolify `marqira-api` environment UI (see `apps/api/.env.example` for the full block). At minimum:
   ```
   MAIL_MAILER=smtp
   MAIL_HOST=smtp.hostinger.com
   MAIL_PORT=465
   MAIL_SCHEME=smtps
   MAIL_USERNAME=noreply@marqira.com
   MAIL_PASSWORD=          # ← paste the real mailbox password HERE in Coolify only
   MAIL_FROM_ADDRESS=noreply@marqira.com
   MAIL_FROM_NAME="MarQira Pulse"
   MARQIRA_ALERTS_ENABLED=true
   MARQIRA_ALERT_EMAIL=ozman.best@gmail.com
   MARQIRA_OFFLINE_ALERT_REPEAT_MINUTES=60
   ```

   > **Security:** never commit the real `MAIL_PASSWORD`. It lives only in the Coolify env UI. The repo keeps it blank.

2. **Run the migration** in the API container Terminal:
   `php artisan migrate --force`
3. **Ensure a queue worker is running.** Alert emails are queued, so they will **not** send unless a worker consumes the Redis queue. In Coolify add (or confirm) a long-running process / extra container running:
   `php artisan queue:work redis --tries=3 --sleep=3`
   Without a worker, alerts silently accumulate in the queue and no email is delivered. (If you prefer no worker, you may switch `QUEUE_CONNECTION=sync`, but that makes the scheduler/heartbeat block on SMTP — not recommended.)
4. **Rebuild config cache** so the new env is picked up:
   `php artisan config:cache`

5. **(Optional) confirm the scheduler is active.** The offline check relies on Laravel's scheduler running every minute (`php artisan schedule:run` via cron or the Coolify scheduler container). This was already required for `marqira:check-stale-sites`; no change here.

Verifying alerts end-to-end (optional)

1. Temporarily set `MARQIRA_OFFLINE_ALERT_REPEAT_MINUTES=2` and lower `MARQIRA_OFFLINE_THRESHOLD` behavior via the heartbeat config if you want a fast test, then `php artisan config:cache`.
2. Stop a test site's heartbeat (deactivate its connector). Within the stale threshold the scheduler marks it offline and you receive the **offline** email; a **repeat** arrives every 2 minutes.
3. Reactivate the connector. On its next heartbeat you receive the **recovery** email and the dashboard shows the site online again.
4. Restore the production repeat interval and re-cache config.

Rollback

- **DB:** `php artisan migrate:rollback` removes the three new columns.
- **Alerts:** set `MARQIRA_ALERTS_ENABLED=false` (and `config:cache`) to disable all alert emails instantly without a redeploy. Offline status detection continues to work.

Increment 4 — Review fixes: durable pairing on delete/reinstall + 1-minute monitor

Commit: `c2b2cf1`
Risk: Low. No migrations, no schema changes, no forced reconnection. One connector change (uninstall behavior + Disconnect cron teardown) and one API change (scheduler cadence + concurrency-safe alerting). Both are reversible.

Two issues found during runbook review are fixed here.

Fix 1 — Pairing now survives a full plugin delete + reinstall

Before: the runbook implied that deleting/uninstalling the connector removed the pairing credentials, so a delete + reinstall would have required a brand-new enrollment code.

After: the pairing credentials (`marqira_site_credentials`) are treated as a **durable connection identity**. The supported lifecycle now works with no new code and no duplicate dashboard site:

```
Deactivate → Delete plugin → Reinstall plugin → Still connected
```

On reinstall the connector finds the existing credentials, re-authenticates with the **same** site UUID/secret, and its per-request cron self-heal automatically restores the heartbeat schedule.

What the uninstaller (`uninstall.php`) now does on plugin delete:
- **KEEPS:** `marqira_site_credentials` (durable pairing) — this is the change.
- **Removes only disposable local state:** the `marqira_connector_settings` option, transient caches (last-heartbeat marker, allowed-IP / Cloudflare caches), the heartbeat cron event (recreated automatically on reinstall), and the `{prefix}marqira_log` security-log table.
- **Never touches:** WordPress Application Passwords or any WP core/user data.

What data remains in WordPress after the plugin is deleted: only the single encrypted option `marqira_site_credentials` (the site UUID, secret and key id, AES-256-GCM encrypted at rest). Everything else the connector created is removed.

The pairing credentials are deleted by exactly two explicit actions — never by ordinary plugin delete/uninstall:
1. The user clicks **Disconnect from MarQira Pulse** inside WordPress (`Marqira_Enrollment::disconnect()`), which now also tears down the heartbeat cron.
2. The site is **removed/revoked from the MarQira dashboard** — the next heartbeat receives `403 site_revoked` and the connector self-disconnects (`Marqira_Heartbeat::handle_revocation()`), clearing credentials and cron.

Fix 2 — Stale-site monitor now runs every minute (honors short repeats)

Before: `marqira:check-stale-sites` ran **every 5 minutes**, so a configured repeat interval shorter than 5 minutes (e.g. the 2-minute test value) could never actually fire on time.

After: the scheduler runs **every minute** (`routes/console.php` → `->everyMinute()->withoutOverlapping()`). The command is **timestamp-driven**: it inspects each site's `last_offline_alert_at` and only sends a repeat once `MARQIRA_OFFLINE_ALERT_REPEAT_MINUTES` has actually elapsed. **Running every minute does not mean emailing every minute** — a 2-, 10-, 30- or 60-minute repeat all work with this single architecture.

Concurrency / idempotency: in addition to `withoutOverlapping()` , both the offline transition and each repeat alert are now issued through an **atomic conditional** `UPDATE` (a DB "claim"). If two scheduler processes ever overlap, exactly one wins the claim and sends; the other sees zero affected rows and sends nothing. No duplicate offline emails, no duplicate repeat emails.

The four independent timing knobs

These are deliberately separate and independently configurable (documented in `apps/api/config/marqira.php`):

#	Knob	Where configured	Test value	Prod value
1	**Heartbeat cadence** — how often each site beats	Connector plugin: `Marqira_Heartbeat::HEARTBEAT_INTERVAL_MINUTES`	2 min	10 min
2	**Offline / stale threshold** — no-beat time before offline	API: `marqira.heartbeat.offline_threshold_minutes`	30 min	30 min
3	**Monitor frequency** — how often the server checks + sends due alerts	API: `routes/console.php` scheduler	every 1 min	every 1 min
4	**Repeat-alert frequency** — how often a still-offline site re-alerts	API env: `MARQIRA_OFFLINE_ALERT_REPEAT_MINUTES`	2 min	60 min

Rule of thumb: heartbeat cadence (1) < offline threshold (2); monitor frequency (3) ≤ repeat frequency (4). Because the monitor now runs every minute, any repeat interval ≥ 1 minute is honored exactly.

Manual steps after redeploy

1. **API redeploy required** (scheduler cadence + command/service changes). **No migration** for this increment.
2. **Confirm the Laravel scheduler runs every minute.** This was already required for Increment 3 (`* * * * * php artisan schedule:run` via cron or the Coolify scheduler container). Nothing new to add — the change is inside the scheduled command, which now self-registers at the 1-minute cadence.
3. **Confirm the queue worker is running** (`php artisan queue:work redis`) — as in Increment 3, alerts are queued and will not send without a worker.
4. **Rebuild config cache** so the updated `config/marqira.php` comments/values and any changed env are picked up:
   `php artisan config:cache`
5. **Connector plugin republish required** (updated `uninstall.php` and `Disconnect` behavior). **No customer reconnection is required** — existing sites keep their credentials and simply gain the delete/reinstall durability.

Can a 2-minute repeat alert actually occur now?

Yes. Set `MARQIRA_OFFLINE_ALERT_REPEAT_MINUTES=2` (and `php artisan config:cache`). With the monitor running every minute, a still-offline site is re-alerted on the first run after each 2-minute interval elapses — no longer blocked by a 5-minute scheduler.

Tests added

- **Connector:** `tests/test-persistent-pairing.php` (21 assertions) — enrolled site survives deactivate → delete (real `uninstall.php`) → reinstall with the same UUID and auto-restored cron; explicit Disconnect clears credentials + cron; dashboard revocation (403) clears credentials + cron. Full connector suite: 94 passing.
- **API:** 5 new cases in `tests/Feature/OfflineAlertTest.php` — every-minute runs never double-send the initial or repeat alert; a 2-minute repeat fires at 3 minutes but not at 1 minute; the atomic claim prevents a concurrent double-send. Full API suite: 106 passing.

Rollback

- **API:** revert `routes/console.php` to `->everyFiveMinutes()` and the `CheckStaleSitesCommand` / `OfflineAlertService` changes. No DB changes to undo.
- **Connector:** re-publishing the previous plugin folder restores the old uninstall behavior; already-paired sites are unaffected (credentials persist).

Increment 5 — WordPress data collection: users, posts, and §26 IP-retention fix

Commit: `caaa680`
Risk: Low. Two new append-only tables (`site_users` , `site_posts`) with standard migrations. One controller fix (§26 IP-retention). Two new HMAC-authenticated endpoints. Connector data-collection class (opt-in, not auto-scheduled yet). All reversible.

What changed (operationally)

§26 IP-retention fix (HeartbeatController): Previously, when a connector sent a heartbeat with `server_ip` omitted (e.g., `SERVER_ADDR` unavailable in a WP-Cron/LiteSpeed context), the controller unconditionally overwrote the site's `server_ip` column with `null` , losing the last known good IP. Now, `server_ip` is only updated when the heartbeat provides a valid one (`$request->filled('server_ip')`). A null or omitted value preserves the existing IP.

New data collection capability: The connector can now collect and ship WordPress user and post/page snapshots to the API for centralized monitoring and analytics. This is **opt-in** (not automatically scheduled in this increment — manual trigger or future scheduling).

- **Users:** `wp_user_id` , `user_login` , `user_email` (may be redacted for privacy), `display_name` , `user_registered` , `roles` , `last_login_at` (if tracked by a plugin), and extensible metadata.
- **Posts/Pages:** `wp_post_id` , `post_type` , `post_status` , `post_title` , `post_date` , `post_modified` , `post_author_id` , `post_author_name` , `guid` , and metadata (categories, tags).

Data is batched (up to 1000 users or posts per call), HMAC-authenticated, and stored append-only in `site_users` / `site_posts` tables. Over time, multiple snapshots of the same `wp_user_id` / `wp_post_id` accumulate, allowing historical tracking of role changes, content edits, etc.

New database tables (migrations `2024_01_04_000001`, `2024_01_04_000002`)

`site_users`:
- Append-only snapshots of WordPress users.
- Key fields: `site_id` (FK), `organization_id`, `snapshot_at`, `wp_user_id`, `user_login`, `user_email`, `display_name`, `user_registered`, `roles` (jsonb), `last_login_at`, `metadata` (jsonb).
- Indexes: `(site_id, snapshot_at)`, `(organization_id, snapshot_at)`, `(site_id, wp_user_id)`.

`site_posts`:
- Append-only snapshots of WordPress posts/pages.
- Key fields: `site_id` (FK), `organization_id`, `snapshot_at`, `wp_post_id`, `post_type`, `post_status`, `post_title`, `post_date`, `post_modified`, `post_author_id`, `post_author_name`, `guid`, `metadata` (jsonb).
- Indexes: `(site_id, snapshot_at)`, `(organization_id, snapshot_at)`, `(site_id, wp_post_id)`, `(site_id, post_type, post_status)`.

New API endpoints (HMAC-authenticated)

- **POST** `/api/v1/sites/users` — receive a batch of user snapshots. Validates `snapshot_at`, `users` array (max 1000), required fields per user (`wp_user_id`, `user_login`).
- **POST** `/api/v1/sites/posts` — receive a batch of post snapshots. Validates `snapshot_at`, `posts` array (max 1000), required fields per post (`wp_post_id`, `post_type`).

Both return `{"success": true, "inserted": N}` on success, 422 on validation failure, 500 on error. Both use the existing HMAC middleware.

New connector class: `Marqira_Data_Collector`

Lives in `includes/class-marqira-data-collector.php`. Provides:
- `collect_users($limit)` — query WordPress users (excluding password hashes), return array.
- `collect_posts($limit)` — query posts/pages (excluding auto-drafts, revisions, trash), return array with categories/tags.
- `ship_users($users)` / `ship_posts($posts)` — HMAC-signed POST to API endpoints.
- `collect_and_ship_all()` — main entry point for periodic collection (not yet scheduled).

Privacy note: user emails are currently included in snapshots. A future phase can redact or hash them based on org privacy settings.

Manual steps after redeploy

1. **Run the migrations** in the API container Terminal:
   `php artisan migrate --force`
   Creates `site_users` and `site_posts` tables.

2. **API redeploy required** (HeartbeatController fix + new endpoints + models). **No environment changes** for this increment.

3. **Connector plugin republish required** (new `Marqira_Data_Collector` class). **No customer action required** — the data-collection methods are available but not automatically invoked yet. A future increment (or manual trigger) will schedule periodic snapshots.

4. **Rebuild config cache** (standard):
   `php artisan config:cache`

Testing the new data collection (optional)

From a connected WordPress site's PHP console or a custom WP-CLI command:

```
require_once WP_PLUGIN_DIR . '/marqira-connector/includes/class-marqira-data-collector.php';
$result = Marqira_Data_Collector::collect_and_ship_all();
var_dump($result); // should show users_collected, posts_collected, users_shipped, posts_shipped
```

Check the API database:

```
SELECT COUNT(*) FROM site_users WHERE site_id = <site_id>;
SELECT COUNT(*) FROM site_posts WHERE site_id = <site_id>;
```

Tests added

- **Connector:** `tests/test-data-collector.php` (16 assertions) — `collect_users()` / `collect_posts()` return properly formatted data; `ship_*()` methods fail when not enrolled; `collect_and_ship_all()` collects correct counts. Full connector suite: **110 passing** (was 94, +16).
- **API:** `tests/Feature/SiteDataCollectionTest.php` (7 tests, 31 assertions) — user/post snapshots received and stored via HMAC; validation of required fields; endpoints reject unauthenticated requests; **§26 IP-retention fix verified:** heartbeat with omitted or null `server_ip` preserves existing IP. Full API suite: **113 passing** (was 106, +7).

Rollback

- **DB:** `php artisan migrate:rollback --step=2` removes `site_users` and `site_posts` tables. No data loss from existing features.
- **API:** revert `HeartbeatController.php`, `routes/api.php`, and delete `SiteDataController.php`. The IP-retention fix is non-breaking, so rolling back re-introduces the bug but doesn't break existing heartbeats.
- **Connector:** re-publish the previous plugin folder (without `class-marqira-data-collector.php`). No impact on existing sites — the class was opt-in and not scheduled.

What's NOT in this increment (future work)

- **Automatic periodic scheduling:** Data collection is available but not yet auto-scheduled via WP-Cron. A future increment will add a recurring event (e.g., daily user/post snapshots) or an on-demand trigger from the dashboard.
- **Privacy controls:** User email redaction/hashing based on org settings.
- **Dashboard UI:** Displaying collected user/post data in the MarQira dashboard (part of the final increment).

Increment 6 — Dashboard UI for Users & Content Data

Commit: `c34245b`
Risk: Low. Dashboard + API endpoint additions; no migrations, no connector changes, no breaking changes.

What changed

This increment surfaces the user and content data (collected in Increment 5) in the MarQira dashboard. Two new tabs are added to the Website Detail page:

1. **Users & Logins** — displays WordPress user snapshots with:
   - Total user count
   - Most recent login summary (user, timestamp, IP)
   - Paginated user table showing: username, email, roles, registration date, last login, login IP

2. **Content** — displays post/page snapshots with:
   - Total posts count
   - Published vs. scheduled posts summary
   - Filter by status (all / published / scheduled)
   - Paginated posts table showing: title, author, status, type, published date, modified date, view link

Both tabs gracefully handle empty states (no data yet) with user-friendly messages.

New API endpoints

Two new dashboard endpoints were added to `SiteController`:

- **GET** `/api/dashboard/sites/{uuid}/users` — returns paginated user snapshots for a site. Uses PostgreSQL `DISTINCT ON (wp_user_id)` to return the most recent snapshot per user. Supports `per_page` parameter (default 50, max 200).
- **GET** `/api/dashboard/sites/{uuid}/posts` — returns paginated post snapshots for a site. Uses `DISTINCT ON (wp_post_id)` for deduplication. Supports `per_page` and optional `status` filter (`publish`, `future`, `draft`).

Both endpoints are protected by:
- Sanctum authentication (dashboard user session)
- Tenant middleware (organization-scoped)
- Site visibility policy (Subscribers see only their own sites; Owner sees all)

New API resources

- `SiteUserResource` — transforms `SiteUser` model to JSON for the dashboard
- `SitePostResource` — transforms `SitePost` model to JSON for the dashboard

New TypeScript types

- `SiteUser` interface matching the API resource structure
- `SitePost` interface matching the API resource structure

Dashboard changes

Files modified:
- `apps/dashboard/src/pages/WebsiteDetail.tsx` — added two new tabs (`Users & Logins`, `Content`) with full UI implementation
- `apps/dashboard/src/types/index.ts` — added `SiteUser` and `SitePost` TypeScript interfaces

UI features:
- Summary cards displaying aggregate counts and most recent activity
- Filterable and paginated data tables
- Responsive design with mobile support
- Empty states for sites with no data yet
- Role badges, status badges with appropriate color coding
- Clickable links to view posts on the WordPress site

Manual steps after redeploy

1. **Dashboard redeploy required** — the React app must be rebuilt and redeployed to show the new tabs.
2. **API redeploy required** — to expose the new `/users` and `/posts` endpoints (though the backend code changes are minimal).
3. **No migrations required** — all database tables already exist from Increment 5.
4. **No connector changes** — this increment is dashboard-only.

Testing

- **API:** Existing test suite remains at **113 passing**. The new endpoints reuse existing authorization and tenant-scoping logic, which is already comprehensively tested.
- **Dashboard:** TypeScript build succeeds with zero errors. Manual testing confirmed:
- Empty states display correctly for sites with no user/post data
- Pagination works correctly
- Filters work correctly (content status filter)
- Data displays correctly when present
- Tenant scoping prevents cross-tenant access

Rollback

- **Dashboard:** redeploy the previous frontend build. The new tabs will disappear but existing tabs remain functional.
- **API:** revert `SiteController.php` and `routes/api.php`. Remove `SiteUserResource.php` and `SitePostResource.php`.
- **No database changes** — tables created in Increment 5 remain intact.

What's included

✅ **Dashboard display of user data** — total count, last login, user table
✅ **Dashboard display of content data** — post/page listings with metadata
✅ **Pagination and filtering** — handle large datasets gracefully
✅ **Empty states** — clear messaging when no data is available yet
✅ **Authorization** — Subscribers see only their own sites' data; Owner sees all
✅ **Production build** — dashboard builds successfully with no TypeScript errors

What's still NOT included (deferred to future work or already complete)

- **Automatic data collection scheduling** — data collection exists but is not yet auto-scheduled (Increment 5 note)
- **Privacy controls** — email redaction/hashing (future enhancement)

- **Real-time updates** — dashboard data refreshes on page load/navigation, not live WebSocket updates
- **Export functionality** — no CSV/PDF export of user/post data yet

---